

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

DP 综合：树形、状压与数位

Advanced DP: Tree, Bitmask and Digit

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 14 日

目录

1. DP 综合概述
2. 树形 DP
3. 状压 DP
4. 数位 DP
5. 总结



目录

1 DP 综合概述

2 树形 DP

3 状压 DP

4 数位 DP

5 总结

三种进阶 DP 的对比

类型	状态形式	转移特点	典型规模
树形 DP	$dp[u][\dots]$ 以节点为主	从子节点向父节点合并	节点数 10^5
状压 DP	$dp[S][\dots]$ 集合为状态	枚举子集或上位状态	$n \le 20$
数位 DP	$dp[\text{pos}][\dots]$ 逐位推进	从高位到低位填数	位数 ≤ 20

三种 DP 都是普通 DP，只是状态的“形状”不同

为什么需要它们

线性 DP 不够用的场景

- 结构是树：状态不是线性下标，而是节点
- 集合关系：状态是一个子集，而不是一段区间
- 数字条件：状态是“前 i 位数字”的某种约束

共同套路

- 先确定状态表示什么
- 再确定状态之间的依赖关系
- 最后确定遍历顺序（后序、按位、按集合大小）

DP 没有新东西：状态、转移、边界、顺序，永远这四步

目录

1 DP 综合概述

2 树形 DP

3 状压 DP

4 数位 DP

5 总结

树形 DP：基本思想

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

核心特征：状态定义在树的**节点**上，转移沿**边**从子节点合并到父节点。

状态一般形式

- $dp[u][0/1/\ldots]$：以 u 为根的子树上的某种最优解
- 第二维表示“u 自己处于什么状态”

遍历顺序：**后序遍历**

- 先递归处理所有子节点
- 再用子节点的答案更新 $dp[u][\cdot]$
- 天然对应 DFS 的“回溯”过程

树形 DP = 在树上做后序遍历，回溯时合并答案

例题一：没有上司的舞会

题目：某公司有 n 名员工，员工之间有上下级关系，构成一棵以老板为根的树。每个员工有一个快乐值 $h[i]$。现在要举办舞会，要求：**如果某个员工参加，他的直接下属都不能参加**。求能获得的最大快乐值总和。

示例

- 3 名员工，1 是 2、3 的上司
- 快乐值 $h = [10,\ 5,\ 8]$（按编号 1、2、3）
- 最优：选 1 和 3，总快乐 $10 + 8 = 18$
- 或选 2 和 3，总快乐 $5 + 8 = 13$

没有上司的舞会：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dp[u][0]$ 表示 u 不参加时子树最大快乐；$dp[u][1]$ 表示 u 参加时子树最大快乐
- 转移：分“u 参加”和“u 不参加”两种情形
- 边界：叶子节点 $dp[u][0] = 0$，$dp[u][1] = h[u]$
- 顺序：后序遍历

转移方程

- u 不参加：子节点可以参加，也可以不参加，取较大者
- $dp[u][0] = \sum_{v \in \text{child}} \max(dp[v][0],\ dp[v][1])$
- u 参加：子节点必须不参加
- $dp[u][1] = h[u] + \sum_{v \in \text{child}} dp[v][0]$

状态里的“取或不取”，是树形 DP 最常见的第二维

没有上司的舞会：代码


```cpp
int n, h[N];
vector<int> adj[N];
int dp[N][2];

void dfs(int u, int parent) {
    dp[u][0] = 0;                    // u 不参加 *
    dp[u][1] = h[u];                 // u 参加 *
    for (int v : adj[u]) {
        if (v == parent) continue;
        dfs(v, u);                   // 后序遍历：先处理子节点
        dp[u][0] += max(dp[v][0], dp[v][1]);   // u 不参加，子节点随意 *
        dp[u][1] += dp[v][0];                  // u 参加，子节点必须不参加 *
    }
}

int main() {
    cin >> n;
    // 读入 h[1..n] 与边，找出根节点 root
```

没有上司的舞会：代码


```
19      int root = 1;                       // 假设根为 1，实际按输入确定
20      dfs(root, 0);
21      cout << max(dp[root][0], dp[root][1]) << endl;
22      return 0;
23  }
```

复杂度 $O(n)$，每个节点只访问一次。

例题二：树的直径

题目：给定一棵 n 个节点的树，求树上**最长路径**的长度。路径长度定义为经过的边数。

示例

- 树结构：$1-2-3-4$，以及 $2-5$
- 最长路径：从 5 到 4，长度为 3（经过 5、2、3、4）
- 或从 1 到 4，长度也是 3

树的直径：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dp[u]$ 表示以 u 为根的子树中，从 u 出发向下的最长路径长度
- 转移：对每个子节点 v，有 $dp[u] = \max(dp[u],\ dp[v] + 1)$
- 直径更新：经过 u 的最长路径，取 u 的两条最长下行路径之和
- 答案：所有节点的“经过 u 的最长路径”取最大值

核心技巧

- 对每个节点，维护“最长”和“次长”两个下行长度
- 直径就是 u 的最长下行 + 次长下行
- 相当于把“拐点在 u 的路径”全部枚举一遍

树的直径 = 所有节点的“最长下行 + 次长下行”的最大值

树的直径：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
vector<int> adj[N];
int ans = 0;                          // 直径 *

// 返回以 u 为根、向下的最长路径长度
int dfs(int u, int parent) {
    int best1 = 0, best2 = 0;         // 最长和次长 *
    for (int v : adj[u]) {
        if (v == parent) continue;
        int d = dfs(v, u) + 1;
        if (d > best1) {
            best2 = best1;
            best1 = d;
        } else if (d > best2) {
            best2 = d;
        }
    }
    ans = max(ans, best1 + best2);  // 经过 u 的最长路径 *
    return best1;
```

树的直径：代码


```
19 }
```

复杂度 $O(n)$；不用两遍 BFS，一次后序遍历即可。

树形 DP 的常见第二维

第二维含义	典型问题	状态
选 / 不选	没有上司的舞会	$dp[u][0/1]$
颜色 / 类别	树上染色、团队分组	$dp[u][c]$
容量 / 花费	树上背包	$dp[u][j]$
距离	树上覆盖、距离统计	$dp[u][d]$

第二维的本质：给节点再贴一个“附加标签”

目录

1 DP 综合概述

2 树形 DP

3 状压 DP

4 数位 DP

5 总结



状压 DP：基本思想

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

核心特征：用二进制整数表示一个**集合**，状态就是“哪些元素已被使用”。

状态的表示

- 集合 $S \subseteq \{0, 1, \ldots, n-1\}$ 编码成整数
- 第 i 位为 1 表示“元素 i 在集合里”
- 状态就是 $dp[S]$ 或 $dp[S][\text{其他维度}]$

典型规模

- $n \leq 20$：状态数 2^n，可以枚举所有子集
- 常见复杂度：$O(2^n \cdot n)$ 或 $O(3^n)$
- 超过 20 就不适合状压了

状压 DP 的本质：把集合当状态，把枚举子集当转移

状压 DP：常用位运算

操作	含义
`S | (1 << i)`	把元素 i 加入集合
`S & ~(1 << i)`	把元素 i 从集合中移除
`S & (1 << i)`	判断元素 i 是否在集合中
`S & (S - 1)`	去掉最低位的 1
`S & (-S)`	取出最低位的 1
`for (T = S; T; T = (T-1) & S)`	枚举 S 的所有非空子集

位运算必须熟练到条件反射；否则状压 DP 写不下去

例题三：旅行商问题

题目：给定 n 个城市和任意两城市之间的距离 $d[i][j]$，从城市 0 出发，每个城市恰好访问一次，最后回到城市 0，求最短总路程。

示例

- $n = 4$，距离矩阵对称
- 一种最优路径：$0 \to 2 \to 1 \to 3 \to 0$，总路程取最小
- 目标是找一条经过所有城市的哈密顿回路

旅行商问题：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dp[S][i]$ 表示已访问集合 S，当前在城市 i 的最短路径
- 转移：从 i 走到未访问的城市 j
- 方程：$dp[S \cup \{j\}][j] = \min\{dp[S][i] + d[i][j]\}$
- 边界：$dp[\{0\}][0] = 0$，其余为 ∞

答案与细节

- 答案：$\min_i\{dp[\text{全集}][i] + d[i][0]\}$
- 状态数：$2^n \times n$；每个状态转移 n 次
- 总复杂度：$O(2^n \cdot n^2)$，$n \le 20$ 时可行

TSP 是状压 DP 的经典入门题，状态设计值得背下来

旅行商问题：代码


```cpp
int n, d[20][20];
int dp[1 << 20][20];

int tsp() {
    const int INF = 1e9;
    int full = (1 << n) - 1;
    // 初始化
    for (int S = 0; S <= full; S++)
        for (int i = 0; i < n; i++) dp[S][i] = INF;
    dp[1][0] = 0;                          // 只访问城市 0, 当前在 0 *

    // 按集合大小顺序枚举 (这里用 S 的整数递增顺序)
    for (int S = 1; S <= full; S++) {
        for (int i = 0; i < n; i++) {
            if (dp[S][i] == INF) continue;
            if (!(S & (1 << i))) continue;
            for (int j = 0; j < n; j++) {
                if (S & (1 << j)) continue;      // j 已访问 *
```

旅行商问题：代码

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19                  int T = S | (1 << j);
20                  dp[T][j] = min(dp[T][j], dp[S][i] + d[i][j]);
21              }
22          }
23      }
24
25      int ans = INF;
26      for (int i = 1; i < n; i++)
27          ans = min(ans, dp[full][i] + d[i][0]);   // 回到起点 *
28      return ans;
29  }
```

例题四：炮兵阵地

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

题目：在 $n \times m$ 的网格上放炮兵，每个炮兵会攻击上下左右各两格的位置。有些格子是山地，不能放炮兵。求最多能放多少个炮兵，使任意两个炮兵都不互相攻击。

示例

- $n = 2$，$m = 3$，地形全为平原
- 一种方案：$(1, 1)$ 和 $(2, 3)$ 各放一个炮兵
- 要求：任意两炮兵的行距大于 2，或列距大于 2

炮兵阵地：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dp[i][S][T]$ 表示第 i 行状态为 S、第 $i-1$ 行状态为 T
- 为什么记两行：炮兵攻击范围覆盖上下各两行，需要看前两行
- 转移：枚举第 $i-2$ 行状态 P，满足三行之间列距约束
- 方程：$dp[i][S][T] = \max\{dp[i-1][T][P] + \text{popcount}(S)\}$

约束条件

- 每行内部：同一行相邻两个炮兵列距要大于 2
- 行与行之间：S、T、P 三行的列距也要满足
- 山地格子：对应的位上不能为 1
- 预处理所有合法状态，能大幅减少枚举量

状压 DP 常用预处理：先把单行合法状态全部筛出来

炮兵阵地：代码框架

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
int n, m;
int terrain[105];                    // 每行的山地掩码
int dp[105][1 << 10][1 << 10];       // 只适用于 m <= 10 *

int count1(int S) {
    int c = 0;
    while (S) { c++; S &= (S - 1); }   // 去掉最低位的 1 *
    return c;
}

int solve() {
    int full = (1 << m) - 1;
    vector<int> valid;
    // 预处理单行合法状态：相邻炮兵至少隔 3 列 *
    for (int S = 0; S <= full; S++) {
        if (S & (S << 1)) continue;          // 相邻列不能都有 *
        if (S & (S << 2)) continue;          // 隔 1 列也不能都有 *
        valid.push_back(S);
```

炮兵阵地：代码框架


```
19      }
20      // ... 三维 DP 转移，判断 S、T、P 之间是否冲突
21      return 0;                                  // 省略具体转移，见下页
22  }
```

预处理合法状态是状压 DP 的常见套路，能砍掉大量无效枚举。

状压 DP 的状态设计套路

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

题型	状态	典型规模
路径 / 哈密顿	$dp[S][i]$ 访问集合 S，当前在 i	$n \le 20$
网格摆棋	$dp[i][S]$ 或 $dp[i][S][T]$	行宽 $m \le 10$
子集划分	$dp[S]$ 集合 S 能否合法划分	$n \le 20$
匹配 / 覆盖	$dp[S][T]$ 两集合的匹配关系	$n \le 15$

状压 DP 的第二维，通常是“当前在哪”或“上一行是什么”

目录

1 DP 综合概述

2 树形 DP

3 状压 DP

4 数位 DP

5 总结



数位 DP：基本思想

核心特征：按数字的**数位**从高到低逐位填数，状态是“已经填到哪一位 + 约束条件”。

状态一般形式

- $dp[\text{pos}][\text{state}][\text{limit}][\text{lead}]$
- `pos`：当前填到第几位
- `state`：跟题目相关的附加信息（和、上一位数字等）
- `limit`：是否紧贴上界
- `lead`：是否处于前导零

典型规模

- 位数 ≤ 20，状态数很小
- 常用记忆化搜索实现，代码短、思路清晰

数位 DP 的本质：一位一位地“枚举”，但用记忆化避免重复

数位 DP：两个关键标记

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

limit 标记

- 表示当前位是否受上界限制
- 若为上界，则当前位最多填到上界对应位
- 若不受限，则当前位可以填 $0 \sim 9$
- 作用是：决定后面是否还有“上界约束”

lead 标记

- 表示当前是否还在“前导零”阶段
- 若还在前导零，当前位可以继续填 0，且“第 1 位实际数字”还没出现
- 作用是：处理“数字位数”相关的问题
- 例如：统计 $[1, N]$ 中数字 d 出现的次数，前导零不算

limit 处理“上界”，lead 处理“位数”，记住这两个就够了

例题五：不含连续 1 的数

题目：给定 N，求 $[0,\ N]$ 范围内，二进制表示中**不含连续两个 1** 的整数个数。

示例

- $N = 5$，**合法数**：0、1、2（10）、4（100）、5（101）
- 3（11）、6（110）、7（111）**都不合法**
- **答案为 5**

不含连续 1 的数：解法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

- 状态：$dp[\text{pos}][\text{prev}][\text{limit}]$，表示第 pos 位，上一位是否为 1
- 转移：枚举当前位填 0 或 1，若上一位是 1，当前位不能填 1
- 边界：填完所有位返回 1
- 顺序：记忆化搜索，从高位往低位

关键点

- 从最高位开始，逐位决定填 0 还是 1
- 只有在 `limit = false` 时才能记忆化（否则状态不通用）
- 本题的前导零不影响结果，可以不使用 `lead`

数位 DP 的记忆化：只在“不受上界约束”时缓存状态

不含连续 1 的数：代码


```cpp
int digit[35];                          // 从高位到低位存储 N 的二进制
int dp[35][2];                          // 只在 limit = false 时使用 *

// pos: 当前位, prev: 上一位是否为 1, limit: 是否紧贴上界
int dfs(int pos, int prev, bool limit) {
    if (pos < 0) return 1;              // 所有位填完, 得到一个合法数 *
    if (!limit && dp[pos][prev] != -1) return dp[pos][prev];

    int up = limit ? digit[pos] : 1;    // 当前位的上界 *
    int res = 0;
    for (int d = 0; d <= up; d++) {
        if (prev == 1 && d == 1) continue;    // 不能出现连续 1 *
        res += dfs(pos - 1, d, limit && (d == up));
    }
    if (!limit) dp[pos][prev] = res;          // 记忆化 *
    return res;
}

```

不含连续 1 的数：代码


```
19 int solve(int N) {
20     if (N < 0) return 0;
21     int len = 0;
22     while (N) { digit[len++] = N & 1; N >>= 1; }
23     // 注意 digit 此时是低位在前，需要反转
24     reverse(digit, digit + len);
25     memset(dp, -1, sizeof(dp));
26     return dfs(len - 1, 0, true);
27 }
```

例题六：数字 1 的个数

题目：给定整数 N，求 $[0,\ N]$ 范围内，所有数字的十进制表示中**数字 1 出现的总次数**。

示例

- $N = 13$，数字 1 出现在 1、10、11、12、13 中
- 1 出现 1 次，10 出现 1 次，11 出现 2 次，12 出现 1 次，13 出现 1 次
- 总共 6 次

数字 1 的个数：解法

四步法

- 状态：$dp[\text{pos}][\text{cnt}][\text{limit}]$，cnt 表示已出现的 1 的个数
- 转移：当前位填 d，若 $d = 1$，$\text{cnt} + 1$
- 边界：填完所有位，返回累计的 cnt
- 顺序：记忆化搜索，从高位往低位

为什么这道题需要 `lead`

- 前导零不是真正的数字 0
- 统计“1 的出现次数”时，前导零不影响（因为不是 1）
- 但如果统计的是“0 的出现次数”，前导零就必须排除
- 本题不需要 `lead`，可以直接省略

是否需要 `lead` 取决于题目是否关心“前导零”

数字 1 的个数：代码


```
int digit[20];
int dp[20][20];                         // pos, cnt *

// pos: 当前位, cnt: 已出现的 1 的个数, limit: 是否紧贴上界
int dfs(int pos, int cnt, bool limit) {
    if (pos < 0) return cnt;            // 所有位填完, 返回累计的 1 的个数 *
    if (!limit && dp[pos][cnt] != -1) return dp[pos][cnt];

    int up = limit ? digit[pos] : 9;
    int res = 0;
    for (int d = 0; d <= up; d++) {
        res += dfs(pos - 1, cnt + (d == 1), limit && (d == up));
    }
    if (!limit) dp[pos][cnt] = res;
    return res;
}

int countDigitOne(int N) {
```

数字 1 的个数：代码


```
19      if (N <= 0) return 0;
20      int len = 0;
21      while (N) { digit[len++] = N % 10; N /= 10; }
22      reverse(digit, digit + len);
23      memset(dp, -1, sizeof(dp));
24      return dfs(len - 1, 0, true);
25  }
```

数位 DP 的通用模板

- 状态：$dp[\text{pos}][\text{state}][\text{limit}]$，必要时加 `lead`
- 转移：在当前位上枚举 0 到上界的每一位
- 返回：填完所有位后返回累计结果
- 记忆化：只在 `limit = false` 时缓存
- 上界：由 `limit` 决定，紧贴时是当前位数字，否则是 9

数位 DP 是一个套路：改的只是 `state` 里装什么

目录

1 DP 综合概述

2 树形 DP

3 状压 DP

4 数位 DP

5 总结



三种 DP 对比速查

类型	状态	转移方向	典型复杂度
树形 DP	$dp[u][\dots]$	子节点到父节点	$O(n)$ 或 $O(nm)$
状压 DP	$dp[S][\dots]$	集合从小到大	$O(2^n \cdot n)$
数位 DP	$dp[\text{pos}][\dots]$	高位到低位	$O(\text{位数} \times \text{状态})$

三种 DP 都是普通 DP，难点在状态设计，不在转移

状态设计心法

- **树形 DP**：状态挂在节点上，第二维装“节点的属性”
- **状压 DP**：状态是集合，第二维装“当前位置”
- **数位 DP**：状态是“已填的位”，附加信息装约束
- 三者共同点：先定“状态是什么”，再想“从哪里转移”
- 三者共同节奏：都是“后序”（子树 / 短集合 / 低位）先算

一切 DP 的本质：状态 + 转移 + 边界 + 顺序

常见易错点

- 树形 DP：递归时忘记传 `parent`，导致无限递归
- 树形 DP：根节点选取错误，多棵树要单独处理
- 状压 DP：忘记判断“状态是否合法”，枚举了大量无效状态
- 状压 DP：n 超过 20 还硬用状压，直接爆空间
- 数位 DP：在 `limit = true` 时也记忆化，答案错误
- 数位 DP：前导零处理不当，多算或漏算
- 三种 DP 共同点：忘记初始化 dp 数组

下一步学什么

- 掌握三种 DP 的典型状态设计和模板
- 能识别题目特征，选择对应的 DP 类型
- 为更复杂的 DP 打基础：
  - 换根 DP：一次遍历求所有节点为根时的答案
  - 状压 DP 的子集枚举优化：$O(3^n)$ 技巧
  - 数位 DP 与计数结合：区间 $[L, R]$ 转前缀相减

DP 之路，从状态设计开始，到状态设计结束

附录：树形 DP 的两种常见套路

套路	特点	典型题目
子树合并	从子节点向上合并答案	没有上司的舞会
换根	先求一个根，再换根	树的直径、距离和
树上背包	合并时做容量 DP	选课、二叉苹果树

换根 DP：第一次 DFS 求子树信息，第二次 DFS 换根修正

附录：状压 DP 常用位运算模板

- 判断 i 是否在 S 中：`S >> i & 1`
- 加入 i：`S | (1 << i)`
- 移除 i：`S & ~(1 << i)`
- 枚举 S 的所有子集：`for (T = S; ; T = (T-1) & S) { ... if (T == 0) break; }`
- 枚举大小为 k 的集合：`for (S = 0; S < (1<<n); S++) if (__builtin_popcount(S) == k) ...`
- 最低位的 1：`lowbit = S & (-S)`
- 去掉最低位的 1：`S & (S - 1)`

位运算写熟，状压 DP 就成功了一半

附录：数位 DP 的记忆化条件

标记	能否记忆化	原因
limit = true	不能	状态与当前上界绑定
limit = false	可以	状态与上界无关，通用
lead = true	不能	状态与“是否还在前导零”绑定
lead = false	可以	状态通用

判断标准：这个状态在不同输入下，后续可能性是否相同

附录：三种 DP 的模板骨架

```cpp
// 树形 DP：后序遍历合并
void dfs(int u, int parent) {
    init(u);                          // 初始化 dp[u]
    for (int v : adj[u]) {
        if (v == parent) continue;
        dfs(v, u);
        merge(u, v);                  // 用子节点更新 dp[u]
    }
}

// 状压 DP：枚举集合与元素
for (int S = 0; S < (1 << n); S++) {
    for (int i = 0; i < n; i++) {
        if (!(S & (1 << i))) continue;
        for (int j = 0; j < n; j++) {
            if (S & (1 << j)) continue;
            int T = S | (1 << j);
            dp[T][j] = min(dp[T][j], dp[S][i] + cost(i, j));
```

附录：三种 DP 的模板骨架


```
        }
    }
}

// 数位 DP：记忆化搜索
int dfs(int pos, int state, bool limit) {
    if (pos < 0) return 1;
    if (!limit && dp[pos][state] != -1) return dp[pos][state];
    int up = limit ? digit[pos] : 9;
    int res = 0;
    for (int d = 0; d <= up; d++)
        res += dfs(pos - 1, trans(state, d), limit && (d == up));
    if (!limit) dp[pos][state] = res;
    return res;
}
```